

SOC MIDTERM PROJECT

Face Recognition using Principal Component Analysis (PCA)

Nitish Kumar 25B4218

June 12, 2026

Contents

1	Introduction	3
1.1	Digital Image	3
2	Key Concepts	3
2.1	Covariance	3
2.2	Covariance Matrix	3
2.3	Eigenvector	4
2.4	Eigenfaces	4
2.5	Mean Face and Mean Subtraction	4
2.6	Weights in Eigenface Space	4
2.7	Euclidean Distance	5
3	Dataset Description	5
4	Principal Component Analysis (PCA)	5
4.1	Step 1: Building the Image Matrix	5
4.2	Step 2: Computing the Mean Face	6
4.3	Step 3: Mean Subtraction (Centring the Data)	6
4.4	Step 4: The Covariance Matrix	6
4.5	Step 5: Eigenvalues and Eigenvectors	7
4.6	Step 6: The Computational Trick	7
4.7	Step 7: Selecting K Eigenfaces	8
4.8	Step 8: Projecting Faces into Eigenface Space	8
4.9	Step 9: Reconstructing Faces from Eigenfaces	8
5	Implementation	9
5.1	Part 1: Loading the Dataset	9
5.2	Part 2: Building the Image Matrix A	9
5.3	Part 3: Applying PCA to Compute Eigenfaces	10
5.4	Part 4: Computing the Weight Matrix	10
5.5	Part 5: Recognising a Known Subject (Subject 39)	11
5.6	Part 6: Recognising an Unknown Subject (Subject 40)	11

6	Results	12
6.1	Test 1: Known Subject (Subject 39)	12
6.2	Test 2: Unknown Subject (Subject 40)	12
6.3	Overall Observations	13
7	Some Insights	13
8	Bonus Learning	13

1 Introduction

Face recognition is one of the most studied problems in computer vision and machine learning. The fundamental challenge is that computers do not perceive faces the way humans do, they process only numbers. To make computer capable of identifying people in images, techniques of **Eigenface** and **Linear Algebra** are fundamentals in face recognition.

1.1 Digital Image

In computer, pictures are represented as a matrix of pixels, with each pixel a particular color coded in some numerical values. and recognising a person from these values is non-trivial because the same face can look drastically different under varying lighting conditions, expressions, or camera angles. Instead of keeping the image as a 2D grid (rows and columns), we 'flatten' it, we take all the rows and lay them end-to-end into one long list of 4096 numbers. This is called vectorizing the image. So, one face image = one vector of length 10304. If you have 300 face images, you have 300 such vectors, each of length 10304.

This project implements face recognition using **Principal Component Analysis (PCA)**, a classical dimensionality reduction technique in Linear Algebra. The specific approach is known as the **Eigenfaces method**. They are the eigenvectors of the covariance matrix of the mean-subtracted pictures of human faces. And the weights used in the weighted sum indeed correspond to the projection of the face picture into each eigenface. The core idea is to transform high-dimensional face images into a compact, low-dimensional representation that preserves identity-relevant information while discarding noise. Recognition is then performed by comparing these compact representations using Euclidean distance.

2 Key Concepts

2.1 Covariance

Variance measures how much one variable spreads around its own mean. Covariance measures how much two variables change together. If when variable X goes up, variable Y also tends to go up, they have positive covariance. If Y tends to go down when X goes up, they have negative covariance.

2.2 Covariance Matrix

A square matrix where entry (i, j) tells you the covariance between dimension i and dimension j of your data. If your data has L dimensions, the covariance matrix is $L \times L$. It summarizes all the relationships between all pairs of dimensions. Here for each face image 10304 pixel dimensions. The covariance matrix would ideally be 10304×10304 , showing how every pixel co-varies with every other pixel across all images.

2.3 Eigenvector

An eigenvector of a matrix is a special direction (vector) such that when you apply the matrix to it, the result is just a scaled version of the same vector. The scaling factor is the corresponding eigenvalue. Formally, if C is the covariance matrix, $\mathbf{v}$ is an eigenvector, and λ is its eigenvalue:

$$C\mathbf{v} = \lambda\mathbf{v}$$

Why does this matter for PCA?

The eigenvectors of the covariance matrix point in the directions of maximum variance in the data. The eigenvalue associated with each eigenvector tells you how much variance exists in that direction.

These eigenvectors are the **Principal Components**.

2.4 Eigenfaces

PCA to a collection of face images, produce principal components (vectors) which have the same dimensions as images, and can be reshape back into image-sized grids and display them as pictures.

Eigenfaces are not real faces. They are abstract 'face patterns' that capture the most important ways in which faces in the dataset differ from each other. The first eigenface captures the biggest source of variation in faces. The second eigenface captures the next biggest source of variation, and so on.

Each eigenface is orthogonal to all the others, making them completely independent of each other.

2.5 Mean Face and Mean Subtraction

Mean Face

The average of all your face images pixel by pixel. The result is a single image called the mean face, which looks like an average blurry face.

Mean Subtraction

Subtract mean face from every image in dataset. This is called mean-centering the data as PCA works on variance around mean. After subtracting the mean, each image vector represents how that particular face differs from the average face. Images that are very similar to the average face will have small values. Faces with unusual features will have large values.

2.6 Weights in Eigenface Space

Different eigenfaces represent different direction by axes (say K) of a coordinate system. Dot product of image vector and each K eigenface produces vector of K numbers. These numbers are called weights which represents the face's coordinates in eigenface space. Higher weight shows more similarity of image with that eigenface (axis).

2.7 Euclidean Distance

Euclidean distance between two vectors is the straight-line distance between them, calculated as the square root of the sum of squared differences.

If that distance is below a threshold, we can say new face belongs to that person. If the distance is above the threshold for all known faces, the face is unrecognized.

3 Dataset Description

The project uses the Olivetti Faces dataset. Photos of the same person were taken at different times, with different lighting, expressions (open/closed eyes, smiling/not smiling), and sometimes glasses. All faces are frontal-view.

Each pixel carries a grayscale intensity value. After loading, scikit-learn normalizes these values to the range $[0, 1]$ instead of the usual $[0, 255]$.

- Number of persons: 40
- Images per persons: 10
- Total images: 400
- Image size: 92×112 pixels
- Grayscale images

Each image contains:

$$112 \times 92 = 10304$$

pixel values.

The dataset is downloaded using:

OpenCV (ORL Databases of Faces from Kaggle)

The dataset is loaded using OpenCV directly from `attface.zip` via Python's `zipfile` package, without extracting to disk.

4 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a mathematical technique that reduces the number of dimensions needed to represent data, while keeping the most important information. In our case, each face image has 10304 pixel values (92×112), which is far too many dimensions to compare directly. PCA finds a small set of special directions in this high-dimensional space that capture the maximum variation across all face images. These special directions, when reshaped into images, are called **Eigenfaces**.

4.1 Step 1: Building the Image Matrix

Each face image of size 92×112 pixels is first **flattened** into a single row vector of length $L = 92 \times 112 = 10304$. If we have M training images, we stack all these row vectors to form the **image matrix** A :

$$A = \begin{bmatrix} - & \mathbf{x}_1^\top & - \\ - & \mathbf{x}_2^\top & - \\ & \vdots & \\ - & \mathbf{x}_M^\top & - \end{bmatrix} \quad A \in \mathbb{R}^{M \times L}$$

where each row $\mathbf{x}_i \in \mathbb{R}^L$ is one flattened face image, M is the number of training images, and $L = 10304$ is the number of pixels per image.

4.2 Step 2: Computing the Mean Face

Before applying PCA, we compute the **mean face** $\bar{\mathbf{x}}$, which is the pixel-by-pixel average of all M training images:

$$\bar{\mathbf{x}} = \frac{1}{M} \sum_{i=1}^M \mathbf{x}_i \quad \bar{\mathbf{x}} \in \mathbb{R}^L$$

The mean face looks like a blurry, averaged version of all faces in the dataset. It represents what a “typical” face looks like in our training set.

4.3 Step 3: Mean Subtraction (Centring the Data)

We subtract the mean face from every training image to produce the **mean-centred** face vectors:

$$\phi_i = \mathbf{x}_i - \bar{\mathbf{x}} \quad \text{for } i = 1, 2, \dots, M$$

Each ϕ_i now represents how face i *differs* from the average face, rather than its absolute pixel values. We replace the rows of A with these centred vectors to get the centred image matrix, still denoted A :

$$A = \begin{bmatrix} - & \phi_1^\top & - \\ - & \phi_2^\top & - \\ & \vdots & \\ - & \phi_M^\top & - \end{bmatrix} \quad A \in \mathbb{R}^{M \times L}$$

This step is essential because PCA measures variance around the mean. Without mean subtraction, the principal components would be dominated by average brightness rather than actual differences between faces.

4.4 Step 4: The Covariance Matrix

The **covariance matrix** C tells us how every pair of pixel dimensions varies together across all training images. Entry $C(i, j)$ is large and positive when pixel i and pixel j tend to be bright or dark together across faces, and near zero when they vary independently. It is defined as:

$$C = \frac{1}{M} A^\top A \quad C \in \mathbb{R}^{L \times L}$$

where $A^\top$ is the transpose of A . Note that C is a square matrix of size $L \times L = 10304 \times 10304$, containing over **106 million entries**. Computing eigenvectors of this matrix directly is computationally infeasible, which leads to the trick described in Step 6.

4.5 Step 5: Eigenvalues and Eigenvectors

For the covariance matrix C , a non-zero vector $\mathbf{u}$ is an **eigenvector**:

$$C \mathbf{u} = \lambda \mathbf{u}$$

Here λ is the **eigenvalue** corresponding to eigenvector $\mathbf{u}$.

- A **large** λ means the data varies a lot in the direction of $\mathbf{u}$, so this direction carries important information.
- A **small** λ means almost no variation in that direction, so it can be safely discarded.

Sorting all eigenvectors by their eigenvalues from largest to smallest gives us the **Principal Components**. Since each eigenvector has $L = 10304$ elements, reshaping it back to 92×112 produces an image. These image-shaped eigenvectors are the **Eigenfaces**.

4.6 Step 6: The Computational Trick

Computing eigenvectors of the full 10304×10304 covariance matrix $C = \frac{1}{M} A A^\top$ is computationally infeasible. However, we can use the following mathematical property.

Instead of computing eigenvectors of $A^\top A \in \mathbb{R}^{L \times L}$, we compute eigenvectors of the much smaller matrix $A A^\top \in \mathbb{R}^{M \times M}$:

$$C' = \frac{1}{M} A A^\top \quad C' \in \mathbb{R}^{M \times M}$$

If $\mathbf{v}_i$ is an eigenvector of C' with eigenvalue λ_i , then the corresponding eigenvector of the full covariance matrix C is obtained by:

$$\mathbf{u}_i = A^\top \mathbf{v}_i$$

For $M = 389$ training images, this replaces a 10304×10304 problem with a 389×389 problem more than $100\times$ smaller. The eigenvalues λ_i are the same for both matrices, so no information is lost.

Matrix	Size	Number of Entries
Full covariance $A^\top A$	10304×10304	≈ 106.2 million
Reduced matrix $A A^\top$	389×389	≈ 0.15 million

4.7 Step 7: Selecting K Eigenfaces

After computing all eigenvectors, we keep only the top K eigenvectors (those with the K largest eigenvalues). These K eigenvectors form the **eigenface matrix** U :

$$U = [\mathbf{u}_1 \ \mathbf{u}_2 \ \cdots \ \mathbf{u}_K] \quad U \in \mathbb{R}^{L \times K}$$

To decide how many eigenfaces to keep, we use the **Explained Variance Ratio** of each component:

$$\text{EVR}_k = \frac{\lambda_k}{\sum_{j=1}^M \lambda_j}$$

The **Cumulative Explained Variance Ratio** for the first K components is:

$$\text{CEVR}(K) = \sum_{k=1}^K \text{EVR}_k = \frac{\sum_{k=1}^K \lambda_k}{\sum_{j=1}^M \lambda_j}$$

A common rule is to choose K such that $\text{CEVR}(K) \geq 0.90$, meaning the selected eigenfaces explain at least 90% of all variation in the training faces. In practice, $K = 50$ is a reasonable default for this dataset.

4.8 Step 8: Projecting Faces into Eigenface Space

Every training face is now projected (encoded) into the K -dimensional eigenface space by computing its **weight vector**:

$$\mathbf{w}_i = U^\top \phi_i \quad \mathbf{w}_i \in \mathbb{R}^K$$

where $\phi_i = \mathbf{x}_i - \bar{\mathbf{x}}$ is the mean-centred face. Each weight w_{ik} tells us how strongly the k -th eigenface contributes to face i . The vector $\mathbf{w}_i$ is the compact K -dimensional fingerprint of face i , replacing the original 10304 pixel values.

4.9 Step 9: Reconstructing Faces from Eigenfaces

A face can be approximately reconstructed from its weight vector using a **linear combination** of eigenfaces:

$$\hat{\mathbf{x}}_i = \bar{\mathbf{x}} + \sum_{k=1}^K w_{ik} \mathbf{u}_k = \bar{\mathbf{x}} + U \mathbf{w}_i$$

With a small K , the reconstruction is blurry but recognisable. With a larger K , the reconstruction becomes more accurate because more eigenfaces are included. This confirms that the eigenfaces genuinely capture the identity information of the face.

5 Implementation

The implementation uses the AT&T face dataset (`attface.zip`), which contains 400 grayscale images across 40 subjects (10 images per subject), each of size 92×112 pixels. The code is divided into six parts, each corresponding directly to a step in the PCA pipeline described earlier.

5.1 Part 1: Loading the Dataset

```
1 faces = {}
2 with zipfile.ZipFile("attface.zip") as facezip:
3     for filename in facezip.namelist():
4         if not filename.endswith(".pgm"):
5             continue
6         with facezip.open(filename) as image:
7             faces[filename] = cv2.imdecode(
8                 np.frombuffer(image.read(), np.uint8),
9                 cv2.IMREAD_GRAYSCALE)
```

The zip file is read directly without extracting to disk. Each `.pgm` file (a grayscale image format) is decoded using OpenCV into a 2D NumPy array of pixel values. All images are stored in a dictionary `faces` where the key is the filename (e.g. `s1/1.pgm`) and the value is the 92×112 pixel array. The filename structure `sX/Y.pgm` identifies subject X and image number Y .

5.2 Part 2: Building the Image Matrix A

```
1 facematrix = []
2 facelabel = []
3 for key, val in faces.items():
4     if key.startswith("s40/"):
5         continue # held out: unknown subject test
6     if key == "s39/10.pgm":
7         continue # held out: known subject test
8     facematrix.append(val.flatten())
9     facelabel.append(key.split("/") [0])
10
11 facematrix = np.array(facematrix)
```

This part constructs the image matrix $A \in \mathbb{R}^{M \times L}$ described in Section 4. Each face image of shape 92×112 is flattened into a row vector of length $L = 92 \times 112 = 10304$ using `val.flatten()`. These row vectors are stacked to form `facematrix`, which is our matrix A .

Two groups of images are deliberately excluded from A to serve as the test set:

- **All 10 images of subject 40** - an entirely unseen subject, used to test recognition of an unknown person.
- **Image 10 of subject 39** - one held-out image of a known subject, used to test recognition of a partially seen person.

After the loop, `facematrix` has shape $(389, 10304)$, i.e. $M = 389$ training images each represented as a vector of $L = 10304$ pixels.

5.3 Part 3: Applying PCA to Compute Eigenfaces

```
1 pca = PCA().fit(facematrix)
2 n_components = 50
3 eigenfaces = pca.components_[:n_components]
```

`PCA().fit(facematrix)` performs all the core PCA steps on the training matrix A :

1. **Mean face:** Computes $\bar{\mathbf{x}} = \frac{1}{M} \sum_{i=1}^M \mathbf{x}_i$, stored in `pca.mean_` as a vector of length $L = 10304$.
2. **Mean subtraction:** Internally centres every row of A as $\phi_i = \mathbf{x}_i - \bar{\mathbf{x}}$.
3. **Reduced covariance:** Forms $C' = \frac{1}{M} A A^\top \in \mathbb{R}^{M \times M}$ instead of the full $C = \frac{1}{M} A^\top A \in \mathbb{R}^{L \times L}$, using the computational trick from Section 4.6.
4. **Eigenvectors:** Computes eigenvectors of C' and maps them back to the full eigenvectors $\mathbf{u}_i = A^\top \mathbf{v}_i$, sorted by descending eigenvalue.

`pca.components_` is the eigenface matrix $U^\top \in \mathbb{R}^{K \times L}$ where each row is one eigenface $\mathbf{u}_k \in \mathbb{R}^L$. We keep only the first $K = 50$ rows, giving:

$$\text{eigenfaces} = U^\top \in \mathbb{R}^{K \times L}, \quad K = 50, \quad L = 10304$$

5.4 Part 4: Computing the Weight Matrix

```
1 weights = eigenfaces @ (facematrix - pca.mean_).T
2 print("Shape of the weight matrix:", weights.shape)
```

This single line projects every training face into the K -dimensional eigenface space. Breaking it down:

- `facematrix - pca.mean_` mean-centres every training image: $\phi_i = \mathbf{x}_i - \bar{\mathbf{x}}$, giving a matrix of shape (M, L) .
- `.T` transposes it to shape (L, M) .
- `eigenfaces @` multiplies $U^\top \in \mathbb{R}^{K \times L}$ by $\Phi \in \mathbb{R}^{L \times M}$, giving the weight matrix:

$$W = U^\top \Phi \in \mathbb{R}^{K \times M}$$

Each column $\mathbf{w}_i \in \mathbb{R}^K$ of W is the weight vector of training face i , computed as:

$$\mathbf{w}_i = U^\top \phi_i$$

The printed shape is $(50, 389)$: 50 eigenface weights for each of the 389 training images. This is the compact representation of all training faces.

5.5 Part 5: Recognising a Known Subject (Subject 39)

```
1 query          = faces["s39/10.pgm"].reshape(1,-1)
2 query_weight   = eigenfaces @ (query - pca.mean_).T
3 euclidean_distance = np.linalg.norm(weights - query_weight, axis
    =0)
4 best_match     = np.argmin(euclidean_distance)
5 print("Best match %s with Euclidean distance %f"
6       % (facelabel[best_match], euclidean_distance[best_match]))
```

The held-out image `s39/10.pgm` is a photo of subject 39, whose other 9 images *are* in the training set. The recognition process mirrors exactly the math from Section 4.8:

1. `query.reshape(1,-1)` flattens the 92×112 test image into a row vector $\mathbf{x}_{\text{test}} \in \mathbb{R}^L$.
2. `eigenfaces @ (query - pca.mean_).T` projects the test face into eigenface space:
 $\mathbf{w}_{\text{test}} = U^T(\mathbf{x}_{\text{test}} - \bar{\mathbf{x}}) \in \mathbb{R}^K$
3. `np.linalg.norm(weights - query_weight, axis=0)` computes the Euclidean distance from $\mathbf{w}_{\text{test}}$ to every training weight vector $\mathbf{w}_j$:

$$d_j = \|\mathbf{w}_{\text{test}} - \mathbf{w}_j\|_2 = \sqrt{\sum_{k=1}^K (w_{\text{test},k} - w_{jk})^2}$$

4. `np.argmin` finds $j^* = \arg \min_j d_j$, the index of the closest training face.

Since the other images of subject 39 are in the training set, the weight vector $\mathbf{w}_{\text{test}}$ is close to those training vectors, and the system correctly identifies the match as belonging to subject 39.

5.6 Part 6: Recognising an Unknown Subject (Subject 40)

```
1 query          = faces["s40/1.pgm"].reshape(1,-1)
2 query_weight   = eigenfaces @ (query - pca.mean_).T
3 euclidean_distance = np.linalg.norm(weights - query_weight, axis
    =0)
4 best_match     = np.argmin(euclidean_distance)
5 print("Best match %s with Euclidean distance %f"
6       % (facelabel[best_match], euclidean_distance[best_match]))
```

The exact same recognition pipeline is applied to an image of subject 40, whose images were *entirely excluded* from the training set. The system still finds the nearest neighbour j^* in eigenface space, but the minimum Euclidean distance d_{j^*} is significantly larger than in the known subject case. This large distance indicates a poor match - the test face lies far from all training faces in eigenface space because none of the training faces belong to this identity.

$$d_{j^*} = \min_j \|\mathbf{w}_{\text{test}} - \mathbf{w}_j\|_2 \gg 0$$

In a production system, a threshold θ would be applied:

$$\text{Decision} = \begin{cases} \text{Subject } j^* & \text{if } d_{j^*} < \theta \\ \text{Unknown} & \text{if } d_{j^*} \geq \theta \end{cases}$$

This experiment demonstrates that the Euclidean distance in eigenface space serves as a natural confidence measure. Small distances indicate reliable recognition and large distances signal an unfamiliar face.

6 Results

The system was tested on two out-of-sample queries after training on $M = 389$ images with $K = 50$ eigenfaces.

6.1 Test 1: Known Subject (Subject 39)

The query image `s39/10.pgm` belongs to subject 39, whose other 9 images were present in the training matrix A . After projecting the query into eigenface space and computing Euclidean distances to all 389 training weight vectors, the system returned:

Query	Best Match	Euclidean Distance
<code>s39/10.pgm</code>	Subject 39	Small ($d_{j^*} \ll \theta$)

The predicted label matched the true label, confirming a correct recognition. This result is expected because the other images of subject 39 are in the training set. Their weight vectors $\mathbf{w}_j$ are close to $\mathbf{w}_{\text{test}}$ in eigenface space, producing a small Euclidean distance d_{j^*} .

6.2 Test 2: Unknown Subject (Subject 40)

The query image `s40/1.pgm` belongs to subject 40, whose *all* 10 images were excluded from training. The system still returned the nearest neighbour, but with a significantly larger distance:

Query	Best Match	Euclidean Distance
<code>s40/1.pgm</code>	Some subject j^*	Large ($d_{j^*} \gg \theta$)

The returned label does not correspond to the true identity. However, the large Euclidean distance d_{j^*} itself is the meaningful signal here. It indicates that the query face lies far from all training faces in the K -dimensional eigenface space. Applying the decision threshold θ :

$$\text{Decision} = \begin{cases} \text{Subject } j^* & \text{if } d_{j^*} < \theta \\ \text{Unknown} & \text{if } d_{j^*} \geq \theta \end{cases}$$

the system would correctly flag this face as **unknown**, demonstrating that Euclidean distance in eigenface space is a reliable confidence measure.

6.3 Overall Observations

- **Correct recognition of known subjects:** When at least some images of a subject are in the training set, the system correctly identifies the person via nearest-neighbour search in the K -dimensional weight space.
- **Detection of unknown subjects:** For a completely unseen subject, the minimum Euclidean distance d_{j*} is noticeably larger than for known subjects. A suitable threshold θ separates the two cases reliably.
- **Dimensionality reduction:** The system reduces each face from $L = 10304$ pixel values to just $K = 50$ weights, a reduction of over 99%, while retaining enough discriminative information for accurate recognition.

7 Some Insights

- I was amazed how this linear algebra technique reduces dimensionality of Image matrix over by 99%.
- Using sklearn and numpy was very helpful in faster computation of matrices of millions of pixels.
- Never thought that by this method of PCA computer recognises people in photos.
- Overall it was a great experience learning this project and applying different concepts of numpy, matplotlib and sklearn that I studied.

8 Bonus Learning

Along with learning from PCA from youtube videos for basic understanding to advanced and in depth learning of mathematical concepts using claude was little bit challenging in starting but as I read them again and again, I got comfortable.

Latex is the one part that i want to highlight. It was quite bit of challenging to download and setup up on my Mac initially, i performed all steps but getting error each time generating pdf. All these packages and syntax were new but after writing 2-3 sections of report it became easy and I am very happy that I learnt LaTeX also while making this project report.

References

- [1] Scikit-Learn Documentation, <https://scikit-learn.org>
- [2] Machine Learning Mastery, <https://machinelearningmastery.com/face-recognition-using-principal-component-analysis/>
- [3] Understanding Mathematics behind PCA, <https://claude.ai/>